

```
#!/usr/bin/env python3
```

```
"""
```

Aether Watcher V0.1 – 大脑显示屏的眼睛

7×24 监控 daemon: 价格 API / RSS 新闻 / 网页变更 → 规则触发 → (可选 LLM 摘要) → 推送 + 告警

架构沿用 aether_crypto 的模式: 常驻循环 + 原子状态落盘 + heartbeat + Streamlit 面板。

目标配置在 watch_targets.toml (stdlib tomlib, 零额外依赖)。

设计原则:

- 轮询和规则判断是确定性代码, LLM 只在「触发后生成人话摘要」这一步进场 (可选)
- 每个目标独立 interval 与独立失败计数, 单目标挂掉不影响其他
- 状态全部落 state/, dashboard 与 daemon 通过文件解耦

依赖: requests (必需); ANTHROPIC_API_KEY (可选, 触发摘要); FIRECRAWL_API_KEY (可选, JS

```
"""
```

```
import os
```

```
import re
```

```
import json
```

```
import time
```

```
import signal
```

```
import hashlib
```

```
import datetime
```

```
import tempfile
```

```
import logging
```

```
import tomlib
```

```
from html.parser import HTMLParser
```

```
from xml.etree import ElementTree
```

```
from typing import Optional
```

```
import requests
```

```
from dotenv import load_dotenv
```

```
load_dotenv()
```

```
# ===== 路径与配置 =====
```

```
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
```

```
STATE_DIR = os.path.join(BASE_DIR, "state")
```

```
os.makedirs(STATE_DIR, exist_ok=True)
```

```
TARGETS_PATH = os.path.join(BASE_DIR, "watch_targets.toml")
```

```
STATE_PATH = os.path.join(STATE_DIR, "watch_state.json")
```

```
EVENTS_PATH = os.path.join(STATE_DIR, "events.json") # 触发记录 (dashboard 自
```

```
HEARTBEAT_PATH = os.path.join(STATE_DIR, "heartbeat.json")
```

```
LOG_PATH = os.path.join(STATE_DIR, "watcher.log")
```

```

PUSHOVER_USER = os.getenv("PUSHOVER_USER", "")
PUSHOVER_TOKEN = os.getenv("PUSHOVER_TOKEN", "")
TELEGRAM_BOT_TOKEN = os.getenv("TELEGRAM_BOT_TOKEN", "")
TELEGRAM_CHAT_ID = os.getenv("TELEGRAM_CHAT_ID", "")
ANTHROPIC_API_KEY = os.getenv("ANTHROPIC_API_KEY", "")
FIRECRAWL_API_KEY = os.getenv("FIRECRAWL_API_KEY", "")
HTTP_TIMEOUT = float(os.getenv("HTTP_TIMEOUT", "15"))
LOOP_SLEEP = int(os.getenv("LOOP_SLEEP", "20"))

logging.basicConfig(level=logging.INFO, format="%(asctime)s - %(levelname)s - %(m
                        handlers=[logging.FileHandler(LOG_PATH), logging.StreamHandle
logger = logging.getLogger("AetherWatcher")
SESSION = requests.Session()
SESSION.headers["User-Agent"] = "Mozilla/5.0 (AetherWatcher/0.1)"

# ===== 原子 IO =====
def atomic_write_json(filepath: str, data):
    tmp = None
    try:
        with tempfile.NamedTemporaryFile(mode="w", dir=os.path.dirname(filepath),
            json.dump(data, f, ensure_ascii=False, indent=2, default=str)
        tmp = f.name
        os.replace(tmp, filepath)
    except Exception as e:
        logger.error(f"原子写入失败 {filepath}: {e}")
        if tmp and os.path.exists(tmp):
            try: os.unlink(tmp)
            except OSError: pass

def safe_read_json(filepath: str, default=None):
    try:
        with open(filepath) as f:
            return json.load(f)
    except (FileNotFoundError, json.JSONDecodeError):
        return {} if default is None else default

# ===== 通知 =====
def notify(message: str, priority: str = "normal"):
    sent = False
    if PUSHOVER_USER and PUSHOVER_TOKEN:
        data = {"token": PUSHOVER_TOKEN, "user": PUSHOVER_USER, "message": messag
        if priority == "critical":
            data.update({"priority": 2, "retry": 60, "expire": 300})

```

```

        try:
            sent = SESSION.post("https://api.pushover.net/1/messages.json", data=
except Exception as e:
            logger.error(f"Pushover 异常: {e}")
if not sent and TELEGRAM_BOT_TOKEN and TELEGRAM_CHAT_ID:
    try:
        SESSION.post(f"https://api.telegram.org/bot{TELEGRAM_BOT_TOKEN}/sendM
                        json={"chat_id": TELEGRAM_CHAT_ID, "text": message[:4096
except Exception as e:
        logger.error(f"Telegram 异常: {e}")

# ===== LLM 摘要 (可选, 仅触发时调用) =====
def llm_summary(context: str) -> Optional[str]:
    if not ANTHROPIC_API_KEY:
        return None
    try:
        r = SESSION.post("https://api.anthropic.com/v1/messages",
            headers={"x-api-key": ANTHROPIC_API_KEY, "anthropic-version": "2023-0
                "content-type": "application/json"},
            json={"model": "claude-haiku-4-5-20251001", "max_tokens": 300,
                "messages": [{"role": "user", "content":
                    "你是监控系统的摘要器。下面是一次触发的原始上下文, 用 2-3 句中文说清:
                    \"幅度/要点、是否值得立即关注。直接输出摘要, 不要客套。\\n\\n\" + contex
                timeout=30)
        if r.status_code == 200:
            return "".join(b.get("text", "") for b in r.json().get("content", []))
            logger.error(f"LLM 摘要失败: HTTP {r.status_code} {r.text[:120]}")
    except Exception as e:
        logger.error(f"LLM 摘要异常: {e}")
    return None

# ===== 抓取器 =====
class _TextExtract(HTMLParser):
    """轻量正文提取: 剥 script/style, 收集可见文本。"""
    SKIP = {"script", "style", "noscript", "svg", "head"}
    def __init__(self):
        super().__init__(); self.parts = []; self._skip = 0
    def handle_starttag(self, tag, attrs):
        if tag in self.SKIP: self._skip += 1
    def handle_endtag(self, tag):
        if tag in self.SKIP and self._skip > 0: self._skip -= 1
    def handle_data(self, d):
        if not self._skip and d.strip(): self.parts.append(d.strip())

```

```

def fetch_price(t: dict) -> Optional[float]:
    """通用 JSON 价格源。json_path 用点路径, 如 'data.amount' 或 'bitcoin.usd'。"""
    try:
        r = SESSION.get(t["url"], timeout=HTTP_TIMEOUT)
        r.raise_for_status()
        node = r.json()
        for key in t["json_path"].split("."):
            node = node[int(key)] if isinstance(node, list) else node[key]
        return float(node)
    except Exception as e:
        logger.error(f"[{t['name']}] 价格抓取失败: {e}")
        return None

def fetch_rss(t: dict) -> Optional[list]:
    """返回 [(id, title, link)], 兼容 RSS2.0 与 Atom。"""
    try:
        r = SESSION.get(t["url"], timeout=HTTP_TIMEOUT)
        r.raise_for_status()
        root = ElementTree.fromstring(r.content)
        ns = {"a": "http://www.w3.org/2005/Atom"}
        items = []
        for it in root.iter("item"):
            # RSS 2.0
            title = (it.findtext("title") or "").strip()
            link = (it.findtext("link") or "").strip()
            guid = (it.findtext("guid") or link or title).strip()
            items.append((guid, title, link))
        if not items:
            # Atom
            for e in root.findall(".//a:entry", ns):
                title = (e.findtext("a:title", "", ns) or "").strip()
                le = e.find("a:link", ns)
                link = le.get("href", "") if le is not None else ""
                eid = (e.findtext("a:id", "", ns) or link or title).strip()
                items.append((eid, title, link))
        return items[:30]
    except Exception as e:
        logger.error(f"[{t['name']}] RSS 抓取失败: {e}")
        return None

def fetch_page_text(t: dict) -> Optional[str]:
    """网页正文。需要 JS 渲染的目标在 toml 里设 render=true (走 Firecrawl API)。"""
    try:
        if t.get("render") and FIRECRAWL_API_KEY:
            r = SESSION.post("https://api.firecrawl.dev/v1/scrape",
                             headers={"Authorization": f"Bearer {FIRECRAWL_API_KEY}"},
                             json={"url": t["url"], "formats": ["markdown"]}, timeout=60)

```

```

        r.raise_for_status()
        return (r.json().get("data") or {}).get("markdown", "")
    r = SESSION.get(t["url"], timeout=HTTP_TIMEOUT)
    r.raise_for_status()
    p = _TextExtract(); p.feed(r.text)
    text = "\n".join(p.parts)
    if t.get("css_select_hint"): # 粗粒度截取: 只看包含锚文本之后的 N 字符
        i = text.find(t["css_select_hint"])
        if i >= 0: text = text[i:i + 4000]
    return text
except Exception as e:
    logger.error(f"[{t['name']}] 页面抓取失败: {e}")
    return None

# ===== 规则引擎 =====
def record_event(target: dict, kind: str, detail: str, summary: Optional[str], pr
    events = safe_read_json(EVENTS_PATH, [])
    if not isinstance(events, list): events = []
    events.append({"time": datetime.datetime.now().astimezone().isoformat(),
                  "target": target["name"], "kind": kind,
                  "detail": detail[:500], "summary": summary})
    atomic_write_json(EVENTS_PATH, events[-500:])
    msg = f"👁️ [{target['name']}] {kind}\n{detail[:300]}"
    if summary:
        msg += f"\n-\n{summary}"
    notify(msg, priority)

def check_target(t: dict, st: dict) -> dict:
    """执行一次目标检查。st 是该目标的持久状态, 返回更新后的 st。"""
    name, kind = t["name"], t["type"]
    priority = t.get("priority", "normal")
    now_iso = datetime.datetime.now().astimezone().isoformat()

    if kind == "price":
        val = fetch_price(t)
        if val is None:
            st["fail"] = st.get("fail", 0) + 1
        else:
            st["fail"] = 0
            base = st.get("baseline")
            st.update({"last_value": val, "last_check": now_iso})
            if base is None:
                st["baseline"] = val
                logger.info(f"[{name}] 基准价 {val}")
            else:

```

```

        change = (val - base) / base
        if abs(change) >= t.get("pct_threshold", 0.10):
            detail = f"价格 {base:.6g} → {val:.6g} ({change:+.1%}, 阈值 ±{t.get('pct_threshold', 0.10)})"
            summary = llm_summary(f"价格监控触发\n目标: {name}\nURL: {t['url']}")
            if t.get("llm_summary") else None
            record_event(t, "价格波动", detail, summary, priority)
            st["baseline"] = val # 触发后重置基准, 避免同一波动反复报警

elif kind == "rss":
    items = fetch_rss(t)
    if items is None:
        st["fail"] = st.get("fail", 0) + 1
    else:
        st["fail"] = 0
        seen = set(st.get("seen_ids", []))
        kw = [k.lower() for k in t.get("keywords", [])]
        fresh = [(i, ti, l) for (i, ti, l) in items if i not in seen]
        if seen: # 首轮只建立基线, 不报警
            hits = [(ti, l) for (i, ti, l) in fresh
                    if not kw or any(k in ti.lower() for k in kw)]
            if hits:
                detail = "\n".join(f"· {ti}\n {l}" for ti, l in hits[:5])
                summary = llm_summary(f"新闻监控触发\n目标: {name}\n关键词: {kw}")
                if t.get("llm_summary") else None
                record_event(t, f"新内容 ×{len(hits)}", detail, summary, priority)
            st["seen_ids"] = list((i for (i, _, _) in items) | seen)[-300:]
            st["last_check"] = now_iso

elif kind == "webpage":
    text = fetch_page_text(t)
    if text is None:
        st["fail"] = st.get("fail", 0) + 1
    else:
        st["fail"] = 0
        h = hashlib.sha256(re.sub(r"\s+", " ", text).encode()).hexdigest()
        old_h = st.get("content_hash")
        st.update({"content_hash": h, "last_check": now_iso})
        if old_h and old_h != h:
            old_text = st.get("content_snippet", "")
            detail = f"页面内容变更 (hash {old_h[:10]} → {h[:10]})"
            summary = llm_summary(
                f"网页变更监控触发\n目标: {name}\nURL: {t['url']}\n"
                f"旧内容片段:\n{old_text[:2000]}\n\n新内容片段:\n{text[:2000]}\n"
                f"请说明实质变化。")
            if t.get("llm_summary") else None
            record_event(t, "页面变更", detail, summary, priority)
            st["content_snippet"] = text[:3000]

```



```

        st.update(check_target(t, st))
        st["_next"] = now + t.get("interval", 300)

    atomic_write_json(STATE_PATH, state)
    atomic_write_json(HEARTBEAT_PATH, {
        "time": datetime.datetime.now().astimezone().isoformat(),
        "ts": time.time(),
        "targets": [{"name": t["name"], "type": t["type"],
                     "last_check": state.get(t["name"], {}).get("last
                     "last_value": state.get(t["name"], {}).get("last
                     "fail": state.get(t["name"], {}).get("fail", 0)}
        })
    time.sleep(LOOP_SLEEP)
except Exception as e:
    logger.error(f"主循环异常: {e}")
    time.sleep(30)

atomic_write_json(STATE_PATH, state)
notify("🛑 Aether Watcher 已停止")
logger.info("已退出")

if __name__ == "__main__":
    Watcher().run()

```